

Vite & Gourmand

Documentation Technique

Architecture · Environnement · Modélisation · Diagrammes · Déploiement

Version 1.0 — 2024

Titre Professionnel Développeur Web et Web Mobile

Sommaire

1. Réflexions initiales technologiques	3
2. Configuration de l'environnement de travail	6
3. Modèle Conceptuel de Données (MCD)	9
4. Diagrammes d'utilisation et de séquence	12
5. Documentation du déploiement	15

1. Réflexions initiales technologiques

Avant de démarrer le développement, une analyse des besoins techniques a été réalisée afin de choisir les technologies les plus adaptées au projet.

1.1 Stack technique imposée par le CDC

Couche	Technologie imposée	Version utilisée
Front-end	HTML5, CSS (Bootstrap), JavaScript	Bootstrap 5.3.0
Back-end	PHP avec PDO	PHP 8.3 + Symfony 7.4
BDD relationnelle	MySQL, MariaDB ou PostgreSQL	MySQL 9.4 (Railway)
BDD NoSQL	MongoDB	MongoDB Atlas 7.0
Déploiement	Fly.io, Heroku, Azure ou Vercel	Render.com

1.2 Pourquoi Symfony plutôt que PHP natif ?

- **Sécurité intégrée** : Protection CSRF, hashage bcrypt, gestion sessions, contrôle d'accès par rôles.
- **Doctrine ORM** : Abstraction BDD évitant les requêtes SQL manuelles pour les opérations courantes.
- **Twig** : Moteur de templates avec échappement automatique des variables (protection XSS native).
- **Mailer Component** : Gestion simplifiée des emails avec support SMTP, Mailtrap, Resend.
- **Structure MVC** : Architecture claire facilitant la maintenabilité et la séparation des responsabilités.

Critère	PHP natif + PDO	Symfony 7
Sécurité CSRF	À implémenter manuellement	■ Intégré
Hashage mdp	password_hash() manuel	■ UserPasswordHasher
Routing	Gestion manuelle	■ Attributs #[Route]
Templates	PHP pur ou include	■ Twig avec échappement
ORM / BDD	PDO manuel	■ Doctrine ORM
Emails	mail() ou PHPMailer	■ Symfony Mailer
Contrôle accès	Sessions manuelles	■ Security Bundle

1.3 Choix de MySQL (Railway) pour la production

Le CDC proposait MySQL, MariaDB ou PostgreSQL. En développement local j'utilise MariaDB 11.8.6. En production, j'ai choisi Railway qui propose MySQL 9.4 pour les raisons suivantes :

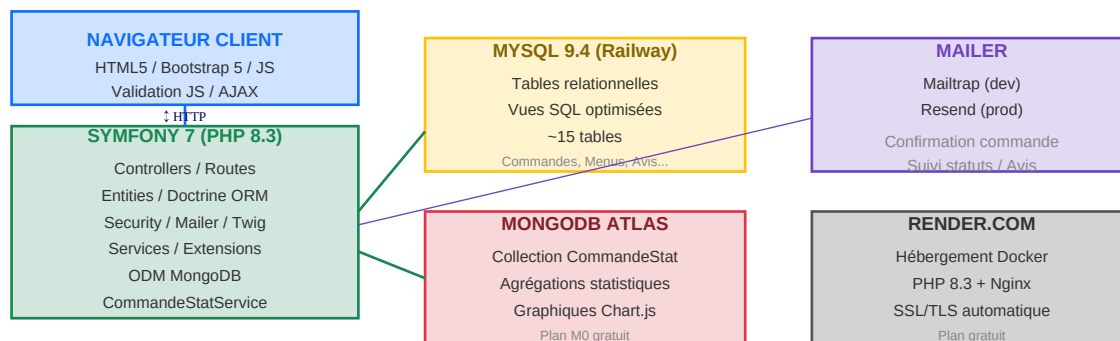
- Compatible avec MariaDB — zéro modification du code Symfony/Doctrine
- Service cloud gratuit (5\$/mois de crédit offert mensuellement)
- Déploiement simple — connexion directe via URL
- Interface web pour administrer la base sans outil externe
- Support des vues SQL utilisées pour optimiser les requêtes fréquentes

Vue	Rôle
v_menus_actifs	Menus actifs avec nombre de plats — page liste et accueil
v_commandes_actives	Commandes non terminées/annulées — dashboard employé
v_avis_publies	Avis validés avec pseudo auteur — page accueil
v_note_moyenne_menu	Note moyenne par menu — vue détaillée menu
v_dashboard_employe	KPI agrégés (compteurs par statut) — dashboard

1.4 Pourquoi MongoDB Atlas pour les statistiques ?

- **Flexibilité du schéma** : Les documents de statistiques peuvent évoluer sans migration.
- **Agrégation puissante** : Pipeline d'agrégation pour calculs complexes (SUM, GROUP BY, SORT).
- **Service cloud gratuit** : Plan M0 gratuit sur MongoDB Atlas — 512 MB suffisants pour les stats.
- **Séparation des préoccupations** : Données transactionnelles dans MySQL, données analytiques dans MongoDB.

1.5 Architecture générale de l'application



2. Configuration de l'environnement de travail

2.1 Système et matériel

Élément	Détail
Système d'exploitation	Fedora Linux
Serveur web (dev)	Symfony CLI (serveur de développement intégré)
PHP (local)	PHP 8.5.6
PHP (prod)	PHP 8.3 (image Docker php:8.3-fpm-bullseye)
Composer	Gestionnaire de dépendances PHP v2
MariaDB (local)	11.8.6 — base de données relationnelle
MySQL (prod)	9.4 — Railway cloud database
MongoDB (prod)	Atlas 7.0 — MongoDB cloud (plan M0 gratuit)
Git	Versioning du code source
Éditeur de code	VS Code / PhpStorm
Navigateur de test	Firefox Developer Edition + Chrome

2.2 Installation du projet en local

Étape 1 — Cloner le dépôt

```
git clone https://github.com/fynorel/vite_gourmand.git cd vite_gourmand
```

Étape 2 — Installer les dépendances

```
composer install
```

Étape 3 — Configurer l'environnement

```
cp .env .env.local # Éditer .env.local avec les paramètres locaux
```

Étape 4 — Importer la structure SQL

```
mariadb -u user -p vite_gourmand < mpd_vite_gourmand.sql
```

Étape 5 — Charger les données de test

```
php bin/console doctrine:fixtures:load --append
```

Étape 6 — Lancer le serveur

```
symfony serve
```

2.3 Variables d'environnement (.env.local)

```
# Environnement APP_ENV=dev APP_SECRET=votre_secret_aleatoire_32_caracteres # Base de données
MariaDB (local) DATABASE_URL="mysql://user:password@127.0.0.1:3306/vite_gourmand
?serverVersion=11.8.6-MariaDB&charset=utf8mb4" # MongoDB (local)
MONGODB_URL="mongodb://127.0.0.1:27017" MONGODB_DB="vite_gourmand" # Mailer (Mailtrap pour le
développement) MAILER_DSN=smtp://identifiant:motdepasse@sandbox.smtp.mailtrap.io:2525
```

2.4 Extensions PHP requises

Extension	Usage	Obligatoire
pdo	Abstraction base de données	■ Oui
pdo_mysql	Connexion MySQL/MariaDB	■ Oui
mongodb	Connexion MongoDB Atlas	■ Oui
intl	Internationalisation Symfony	■ Oui
mbstring	Manipulation chaînes multi-octets	■ Oui
xml	Traitement XML	■ Oui
zip	Compression fichiers	■ Oui
opcache	Cache bytecode PHP (performance)	■ Oui

3. Modèle Conceptuel de Données (MCD)

Le MCD présente les entités principales de la base de données MySQL et leurs relations. La base comprend 15 tables organisées autour de 4 domaines fonctionnels.

3.1 Entités et attributs principaux

UTILISATEUR ■ id_utilisateur (PK) - nom - prenom - mail (UK) - gsm - adresse - mdp_hash - role (ENUM) - actif - date_creation	MENU ■ id_menu (PK) - titre - description - theme - regime - nb_personnes_min - prix - stock - conditions - actif - date_creation
PLAT ■ id_plat (PK) - nom - type - description - actif	COMMANDE ■ id_commande (PK) ■ id_utilisateur (FK) ■ id_menu (FK) - nb_personnes - adresse - date_prestation - prix_menu - reduction - frais_livraison - prix_total - statut - date_creation
AVIS ■ id_avis (PK) ■ id_commande (FK) ■ id_utilisateur (FK) - note - commentaire - statut - date_creation	ALLERGENE ■ id_allergene (PK) - nom (UK)
IMAGE_MENU ■ id_image (PK) ■ id_menu (FK) - url - alt - ordre	HISTORIQUE_STATUT ■ id_historique (PK) ■ id_commande (FK) ■ id_utilisateur (FK) - statut - changed_at - commentaire

3.2 Relations entre entités

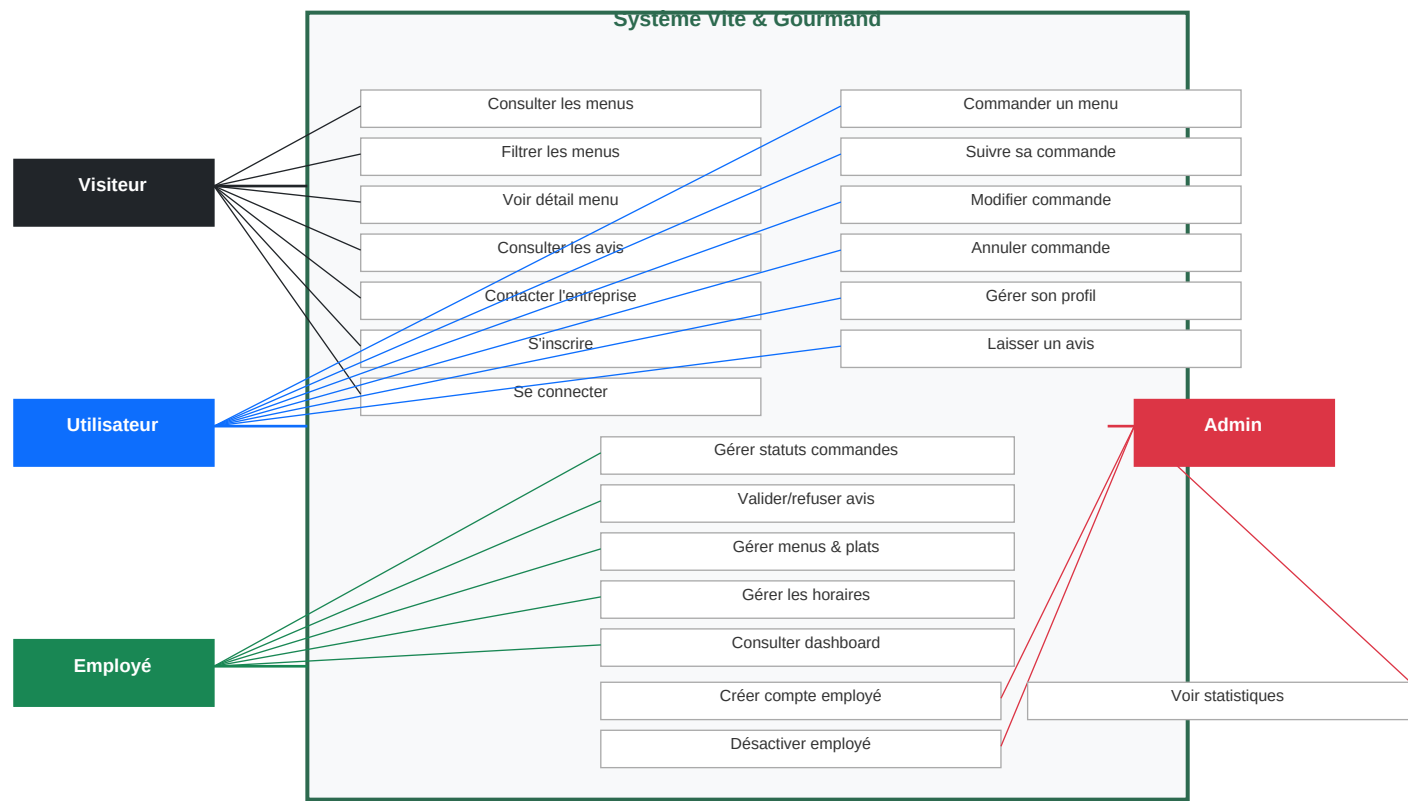
Relation	Type	Description
UTILISATEUR → COMMANDE	1,N	Un utilisateur peut passer plusieurs commandes
MENU → COMMANDE	1,N	Un menu peut être commandé plusieurs fois
MENU ↔ PLAT	N,N	Un menu contient plusieurs plats (table menu_plat)
PLAT ↔ ALLERGENE	N,N	Un plat peut avoir plusieurs allergènes (table plat_allergene)
MENU → IMAGE_MENU	1,N	Un menu peut avoir plusieurs images (galerie)
COMMANDE → AVIS	1,1	Une commande terminée peut avoir au maximum un avis
COMMANDE → HISTORIQUE_STATUT	1,N	Chaque changement de statut est tracé dans l'historique
ENTREPRISE → HORAIRE	1,7	Une entreprise a 7 horaires (un par jour de la semaine)

3.3 Document MongoDB — CommandeStat

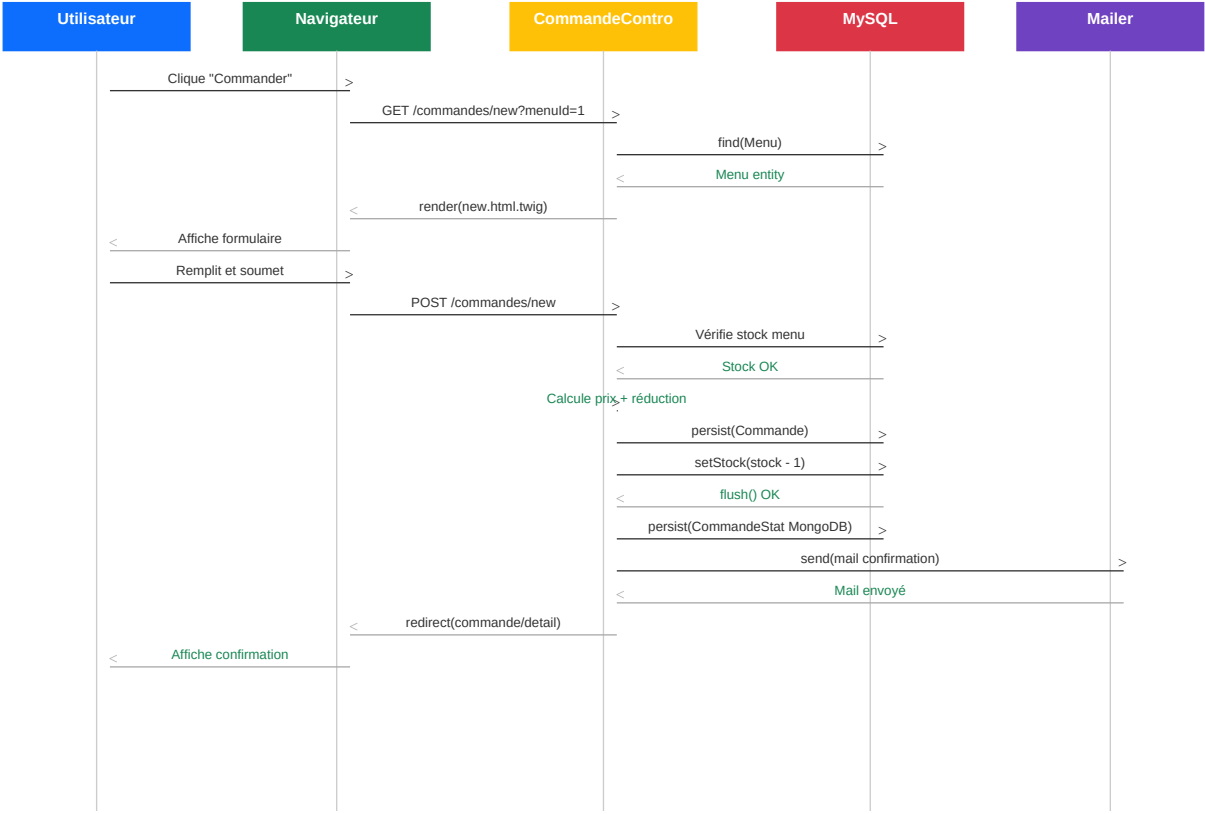
```
{ "_id": ObjectId("..."), "menuId": 1, "menuTitre": "Menu Noël Prestige", "date":
ISODate("2024-12-15T14:30:00Z"), "nombreCommandes": 1, "prixTotal": "320.00", "dateCreation":
ISODate("2024-12-15T14:30:00Z") }
```


4. Diagrammes d'utilisation et de séquence

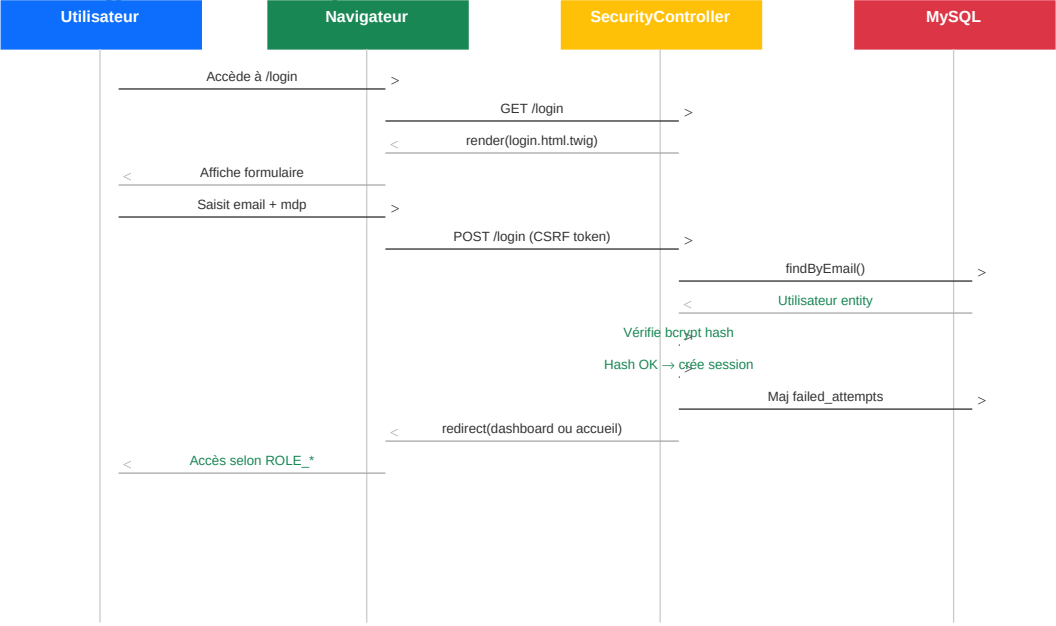
4.1 Diagramme des cas d'utilisation



4.2 Diagramme de séquence — Passage d'une commande



4.3 Diagramme de séquence — Authentication



5. Documentation du déploiement

Cette section décrit la démarche et les étapes suivies pour déployer l'application Vite & Gourmand en production. L'application est accessible à l'adresse : <https://vite-gourmand-oef6.onrender.com>

5.1 Infrastructure de production

Service	Rôle	Plan	URL
Render.com	Hébergement application PHP/Docker	Gratuit	vite-gourmand-oef6.onrender.com
Railway	Base de données MySQL 9.4	Hobby (5\$/mois offerts)	kodama.proxy.rlwy.net
MongoDB Atlas	Base de données NoSQL stats	M0 Gratuit (512 MB)	vite-gourmand.ja6mibm.mongodb.net
Mailtrap	Envoi d'emails (dev/test)	Gratuit	sandbox.smtp.mailtrap.io

5.2 Architecture Docker

L'application est conteneurisée avec Docker. Le conteneur intègre PHP-FPM, Nginx et Supervisor dans une seule image basée sur php:8.3-fpm-bullseye.

Structure des fichiers Docker :

```
vite_gourmand/ ■■■ Dockerfile ← Image Docker (PHP 8.3 + Nginx + Supervisor) ■■■  
.dockerignore ← Fichiers exclus du build ■■■ docker/ ■■■ entrypoint.sh ← Script de démarrage  
(cache Symfony + supervisord) ■■■ php/ ■■■ php.ini ← Configuration PHP production ■ ■■■  
opcache.ini ← Configuration OPcache ■■■ nginx/ ■ ■■■ nginx.conf ← Configuration Nginx  
globale ■ ■■■ default.conf ← VHost Symfony (port 8080) ■■■ supervisor/ ■■■ supervisord.conf  
← Gestion processus (nginx + php-fpm)
```

5.3 Étapes de déploiement

Étape 1 — Préparation de la base de données MySQL (Railway)

- Créer un compte sur railway.app et créer un nouveau projet MySQL
- Récupérer l'URL de connexion publique depuis l'onglet 'Connect' → 'Public Network'
- Importer la structure SQL depuis le terminal local :

```
mariadb -h kodama.proxy.rlwy.net -u root -pMOTDEPASSE --port 50484 --protocol=TCP --skip-ssl  
vite_gourmand < mpd_vite_gourmand.sql  
  
mariadb -h kodama.proxy.rlwy.net -u root -pMOTDEPASSE --port 50484 --protocol=TCP --skip-ssl  
vite_gourmand < seed_vite_gourmand.sql
```

Étape 2 — Configuration MongoDB Atlas

- Créer un compte sur mongodb.com/atlas
- Créer un cluster gratuit M0 (région EU — Frankfurt)
- Créer un utilisateur de base de données avec mot de passe
- Récupérer la connection string : mongodb+srv://user:password@cluster.mongodb.net/
- Dans Network Access → ajouter 0.0.0.0/0 pour autoriser toutes les IPs (nécessaire pour Render)

Étape 3 — Déploiement sur Render

- Créer un compte sur render.com et connecter le compte GitHub
- Créer un nouveau Web Service → sélectionner le dépôt vite_gourmand

- Render détecte automatiquement le Dockerfile et la branche main
- Sélectionner le plan Free (0\$/mois) et la région Frankfurt (EU Central)
- Configurer les variables d'environnement (voir section 5.4)
- Cliquer sur 'Deploy Web Service'

Étape 4 — Création du compte administrateur

Le compte administrateur ne peut pas être créé depuis l'application. Il doit être inséré directement en base de données avec un mot de passe hashé en bcrypt :

```
# Générer le hash bcrypt php -r "echo password_hash('MonMotDePasse', PASSWORD_BCRYPT, ['cost' => 12]);"

# Insérer en base INSERT INTO utilisateur (nom, prenom, mail, mdp_hash, role, actif, failed_attempts, date_creation) VALUES ('NOM', 'Prenom', 'admin@email.fr', 'HASH', 'ADMINISTRATEUR', 1, 0, NOW());
```

5.4 Variables d'environnement Render

Les variables d'environnement sont configurées dans l'interface Render → Environment → Environment Variables :

Variable	Valeur	Description
APP_ENV	prod	Environnement Symfony production
APP_SECRET	(généré)	Clé secrète Symfony (openssl rand -hex 16)
DATABASE_URL	mysql://root:PWD@host:port/vite_gourmand	URL de la base de données
MONGODB_URL	mongodb+srv://user:pwd@cluster.mongodb.net	URL de MongoDB Atlas
MONGODB_DB	vite_gourmand	Nom de la base MongoDB
MAILER_DSN	smtp://...@sandbox.smtp.mailtrap.io:2525	DSN Mailtrap

5.5 Difficultés rencontrées et solutions

- **Extension MongoDB PHP** : La compilation de l'extension MongoDB via PECL échouait sur Alpine Linux à cause du symbole manquant 'strncpy'. Solution : utiliser l'image php:8.3-fpm-bullseye (Debian) à la place d'Alpine, et la version mongodb-1.15.3 compatible avec cette image.
- **Packages PHP 8.4 requis** : Plusieurs packages Doctrine requéraient PHP 8.4 dans le composer.lock alors que Docker utilisait PHP 8.3. Solution : mettre à jour le composer.lock avec --ignore-platform-reqs localement.
- **Scripts Composer en production** : Le script cache:clear de Symfony échouait pendant le build Docker car bin/console n'était pas encore disponible. Solution : copier le code source avant composer install et utiliser --no-scripts.
- **Mapping des rôles Symfony** : Les rôles en base sont ADMINISTRATEUR/EMPLOYE/UTILISATEUR mais Symfony exige le préfixe ROLE_. Solution : implémenter un match() dans getRoles() pour mapper les valeurs ENUM vers les rôles Symfony.
- **Hash mot de passe tronqué** : L'UPDATE SQL du mot de passe tronquait le hash bcrypt à cause des caractères '\$' interprétés par le shell. Solution : utiliser une variable bash pour stocker le hash avant l'UPDATE.
- **PlanetScale non gratuit** : PlanetScale a supprimé son plan gratuit. Alternative : Railway avec 5\$/mois de crédit offert.

5.6 Accès à l'application déployée

Rôle	Email	Mot de passe
Administrateur	jose@vite-gourmand.fr	[Communiqué séparément]
Employé	julie@vite-gourmand.fr	[Communiqué séparément]
Utilisateur	[À créer via l'inscription]	[Défini lors de l'inscription]

■ Les mots de passe exacts des comptes de démonstration sont communiqués dans le manuel d'utilisation.

